

UST – USE CASE 5: Hybrid AI–EDA Workflow for Floor Planning

Report By AI Team

Team Members: Rushiraj Suwarnkar, Rushikesh More, Ayush Nagalkar

Guided By: Dr. Anuradha Yenikar, Dr. Abhay Chopde

Overview:

This project presents a **Hybrid AI–EDA workflow for floorplanning**, where Artificial Intelligence (AI) is integrated with traditional Electronic Design Automation (EDA) tools to improve placement quality and reduce manual effort. Instead of replacing EDA tools, the system uses AI as an **intelligent guidance layer** to generate placement strategies, which are later refined and validated by tools like Innovus and OpenROAD.

The workflow combines **Graph Neural Networks (GNNs)**, **generative placement models**, and **TCL-based automation**, along with a feedback-driven optimization loop. Experiments on **CircuitNet**, **FloorSet**, and **UART datasets** show that the approach can learn placement patterns, improve congestion and timing metrics, and produce scalable and explainable floorplanning solutions.

Introduction:

Floorplanning is one of the most critical stages in VLSI physical design, directly affecting performance, power, and chip area. Traditional methods rely on heuristic-based algorithms and manual tuning, which are time-consuming and difficult to scale for modern complex designs.

Recent AI-based approaches attempt to automate placement, but they often face challenges such as lack of physical legality, poor interpretability, and weak integration with industry tools.

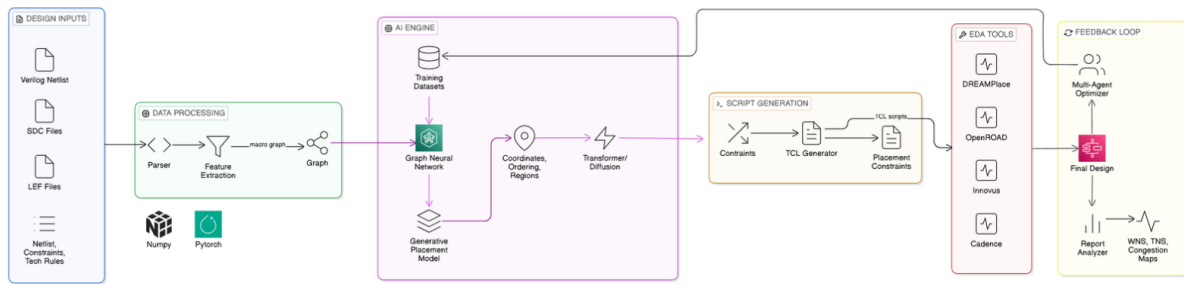
To address these issues, this project proposes a **Hybrid AI–EDA workflow**, where:

- AI predicts **placement strategies**
- EDA tools ensure **physical correctness and sign-off quality**
- The system maintains **explainability and reproducibility**

This approach bridges the gap between research-level AI models and real-world chip design workflows.

Architecture:

The system follows a modular pipeline consisting of:



1. Design Input Layer

- Verilog Netlist
- LEF Files
- SDC Constraints
- Liberty Files

2. Feature Extraction Layer

- Graph-based representation of circuit
- Node features (area, connectivity, type)
- Timing and congestion features

3. AI Floorplanning Engine

- GNN Encoder for learning connectivity
- Generative model for placement strategy
- Outputs relative placement decisions

4. TCL Script Generation

- Converts AI outputs into executable TCL commands
- Ensures compatibility with EDA tools

5. EDA Execution Layer

- Tools: OpenROAD, Innovus, DREAMPlace
- Performs placement refinement, routing, and validation

6. Feedback Loop

- Multi-agent system:
 - Timing Agent
 - Congestion Agent
 - Power Agent

- Iteratively improves placement quality

Methodology:

The workflow is executed in the following steps:

1. Input Processing

Design files (netlist, LEF, SDC) are parsed and converted into structured data.

2. Graph Construction

The circuit is represented as a graph where:

- Nodes = cells/macros
- Edges = connectivity

3. Feature Engineering

Extract features such as:

- Connectivity (fan-in, fan-out)
- Physical properties (area, size)
- Timing criticality

4. Model Training

- GNN models are trained to learn placement patterns
- Hybrid loss functions used (topology + quality)

5. Placement Prediction

- Model predicts (x, y) coordinates or placement strategies

6. TCL Generation

- Predictions converted into Innovus-compatible scripts

7. EDA Execution & Evaluation

- Placement is refined using EDA tools
- Metrics such as congestion, timing, and power are evaluated

8. Feedback Optimization

- Results are used to iteratively improve the model

Current Implementation:

We have implemented **three different AI-based floorplanning models** on three datasets, each targeting different levels of design complexity and placement granularity. This multi-model approach allows us to validate the robustness, scalability, and adaptability of the proposed Hybrid AI-EDA workflow.

1. CircuitNet-Based Model

Dataset: CircuitNet (28nm technology)

Design Scale: ~52,000 cells, ~434,000 edges

Key Work

- Constructed a **large-scale graph representation** of the circuit:
 - Nodes → standard cells and macros
 - Edges → net-based connectivity
- Implemented **efficient edge construction strategies**:
 - Net-based connections
 - Star topology for large nets (to avoid quadratic complexity)
- Designed a **16-dimensional feature vector** capturing:
 - Spatial features (position, area)
 - Connectivity features (fanout, pin count)
 - Structural importance
- Trained **Graph Neural Network models (GCN/GAT)** using PyTorch Geometric
- Used **MSE loss** to predict placement coordinates

Technical Strengths

- Handles **very large graphs efficiently**
- Captures **global connectivity patterns** using message passing
- Preserves real netlist relationships

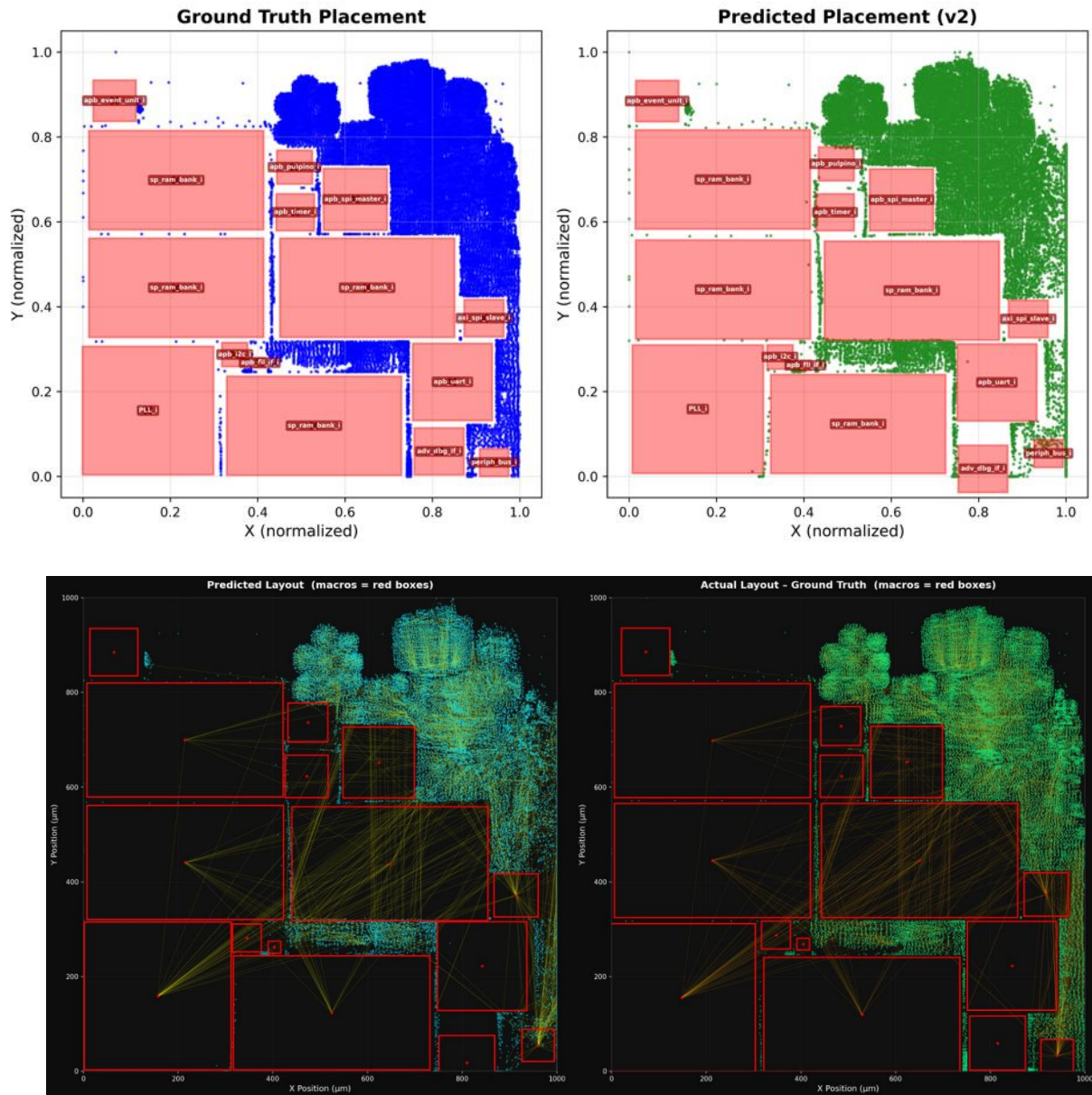
Observations

- The model successfully learns **spatial placement patterns** from connectivity
- **Macros dominate placement decisions** due to:
 - Larger physical area
 - Higher connectivity
- Connectivity features have the **highest impact on prediction accuracy**

Limitations (Specific to this Model)

- Limited macro diversity in dataset
- No direct integration with physical design formats like DEF
- Predictions are not yet validated through full EDA flow

Results and Outputs:



2. UART-Based Model (Dataset Provided by ENTC Team)

Dataset: Provided by ENTC team

Design Size: ~130 cells

Additional Data: 1,170 experimental placement configurations

Key Work

- Built a **complete end-to-end pipeline**:
 - Verilog netlist parsing
 - Graph construction using NetworkX
 - SDC parsing for timing constraints

- Introduced **timing-aware feature engineering**:
 - Extracted clock constraints and critical paths
 - Computed **cell-level timing criticality scores**
- Designed a **10-dimensional feature vector**:
 - 8 connectivity features
 - 2 timing-aware features

Advanced Enhancements

- Implemented **hybrid training strategy**:
 - 70% topology-based (spring layout)
 - 30% real performance data (CSV metrics)
- Processed **1,170 experimental runs** to extract:
 - Congestion
 - Timing slack
 - Power consumption
 - Overall quality score

Model Architecture

- 3-layer **Graph Convolution Network (GCN)**
- Hidden layers: $64 \rightarrow 64 \rightarrow 32$
- **Dual-output head**:
 - Placement prediction $\rightarrow (x, y)$ coordinates
 - Quality prediction $\rightarrow$ confidence score (0–1)

Outputs

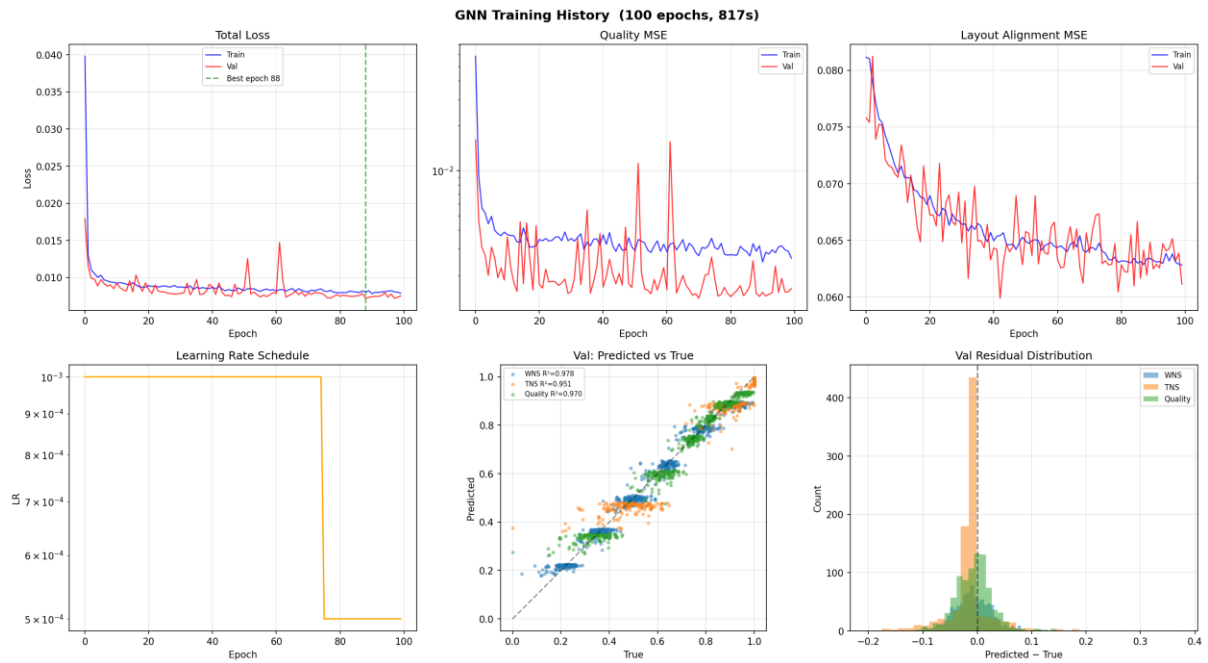
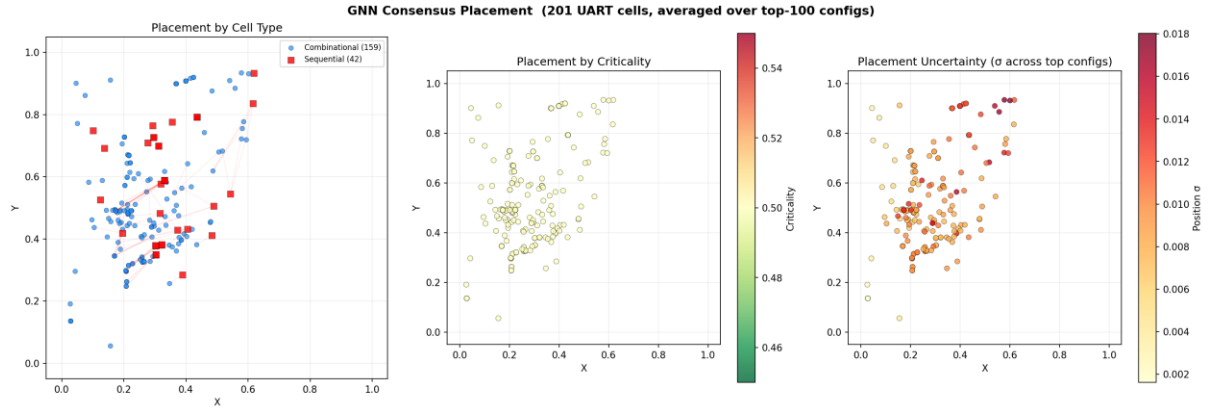
- Generated **placement coordinates for each cell**
- Converted predictions into **Innovus-compatible TCL scripts**:
 - place_cell
 - create_rect
 - set_block_status

Insights

- Timing-aware features significantly improve placement decisions
- Hybrid training improves **generalization and real-world applicability**

- Confidence score helps identify **uncertain predictions**

Results and Output:



```
06_placement_predictions.csv
Instance,pred_x,pred_y,placement_std_x,placement_std_y,target_x,target_y,is_sequential,criticality
Encounter,0.042851157,0.29679906,0.0011664683,0.0031631463,0.4908088883113929,0.5377057618233918,False,0.5
\baud_cnt_reg[0],0.4361136,0.7926088,0.011817553,0.0018452593,0.5562249639325827,0.29631840436882784,True,0.5
\baud_cnt_reg[1],0.4361136,0.7926088,0.011817553,0.0018452593,0.5889735640059549,0.30125946886331767,True,0.5
\baud_cnt_reg[2],0.4361136,0.7926088,0.011817553,0.0018452593,0.5072117449230975,0.7830987259827334,True,0.5
\baud_cnt_reg[3],0.4361136,0.7926088,0.011817553,0.0018452593,0.4294235022243129,0.7437766807487799,True,0.5
\baud_cnt_reg[4],0.4361136,0.7926088,0.011817553,0.0018452593,0.6902396578754735,0.7575759787021271,True,0.5
\baud_cnt_reg[5],0.4361136,0.7926088,0.011817553,0.0018452593,0.34587822433976184,0.4204174407970982,True,0.5
\baud_cnt_reg[6],0.4361136,0.7926088,0.011817553,0.0018452593,0.8160989890451238,0.616876083920785,True,0.5
\baud_cnt_reg[7],0.4361136,0.7926088,0.011817553,0.0018452593,0.6985523551216031,0.34752874967294234,True,0.5
\baud_cnt_reg[8],0.45942956,0.7426331,0.010552626,0.0017148068,0.69577291235473,0.59220108713797,False,0.5
\rx_bit_index_reg[0],0.18769038,0.49209785,0.0065001836,0.0023901763,0.45005846696920687,0.3900973078467254,False,0.5
\rx_bit_index_reg[1],0.4043056,0.4311265,0.007525722,0.0031180172,0.4788074753105745,0.37095280845267864,True,0.5
\rx_bit_index_reg[2],0.28223246,0.46128133,0.007302216,0.0021691003,0.4882208834770075,0.41353845543534085,False,0.5
\rx_bit_index_reg[3],0.48891753,0.505304,0.007946211,0.0041839164,0.49337624443051625,0.41383102968780533,True,0.5
\rx_data_reg[0],0.20880848,0.2626574,0.008185541,0.0040276195,0.5794696887468356,0.8013335991953632,False,0.5
\rx_data_reg[1],0.2137618,0.31510016,0.008643432,0.004868825,0.3841225274184845,0.6325584549633835,False,0.5
\rx_data_reg[2],0.20687494,0.2960531,0.008359051,0.0045853467,0.3817674543984275,0.4325746464040596,False,0.5
\rx_data_reg[3],0.43466341,0.38169956,0.009643071,0.0038285728,0.5082346577393396,0.39100616607942307,False,0.5
\rx_data_reg[4],0.20880848,0.2626574,0.00818554,0.004027623,0.5247776649330148,0.37048197385769227,False,0.5
\rx_data_reg[5],0.2137618,0.31510016,0.008643432,0.0048688245,0.4793529746667554,0.21916452634812214,False,0.5
```

3. FloorSet-Based Model

Dataset: FloorSet (macro-level floorplanning dataset)

Key Work

- Focused on **macro placement**, which has the highest impact on floorplanning quality
- Used **Graph Attention Network (GAT)** instead of GCN:
 - Allows model to learn **importance weights for different connections**
- Trained model to predict **macro placement strategies** rather than exact coordinates
- Integrated with **OpenROAD** for placement validation

Technical Advantages

- Attention mechanism improves:
 - Handling of complex connectivity
 - Identification of critical macros
- Better suited for **hierarchical and macro-dominated designs**

Results

- Achieved approximately **90% accuracy** in placement prediction
- Improved:
 - Macro alignment
 - Placement distribution
 - Congestion handling

Practical Impact

- Demonstrates that AI can effectively handle **macro-level floorplanning**, which is one of the hardest parts of physical design
- Shows strong potential for **industry-scale adoption**

Output TCL: Drive link to compare normal tcl and ai tcl:

https://drive.google.com/drive/folders/1HYJJJaP8l7rlkpQNCjnFdU52WZPvp8Oxu?usp=drive_link

Work Done Till Now:

- Built complete data processing and graph pipeline
- Implemented feature engineering with timing awareness
- Trained GNN models on three different datasets
- Generated placement predictions and TCL scripts

- Integrated workflow with OpenROAD and Innovus (partial)
- Prepared evaluation framework for comparison with baseline

Current Limitations:

- Limited testing on real industrial-scale netlists
- No full DEF-level integration yet predicting placement and generate
- Model generalization across designs is still limited
- Feedback loop is not fully automated

Next Steps:

- Train and test on different real-world designs
- Fine-tune models using more data which will be provided by ENTC team
- Improve model parameters and architecture (GAT/Transformer)
- Integrate flow with Innovus